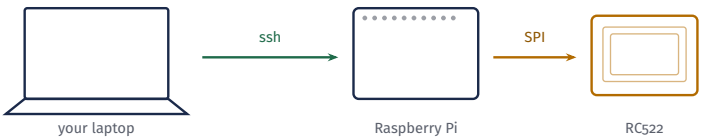
THE MIDNIGHT BREWER · VOLUME ZERO

Setting Everything Up

From a blank SD card to a Raspberry Pi that answers over SSH, builds Java, and talks to an NFC reader.



Read this one first

Revision July 26, 2026

The Midnight Brewer, Volume Zero: Setting Everything Up

This is the prequel. It assumes you have never used a Raspberry Pi, never generated an SSH key, and have never run `mvn` in your life. It ends with a Pi you can reach by name, a project that builds, and an NFC reader that answers.

Volume One — *Building a Contactless Payment Terminal From Scratch* — picks up exactly where this stops, and is where you start writing code.

Every command in this book is meant to be typed. Where output is shown, it is what you should actually see. Where it differs, the troubleshooting appendix is indexed by symptom.

Contents

Before you start xi

I Your Machine 1

1 What you need 3

1.1 The shopping list . . . 3
1.2 Which Pi? . . . 4
1.3 Wires: shorter than you think . . . 4
1.4 What you do not need . . . 5

2 Your laptop 7

2.1 A package manager first . . . 7
2.2 Java 21 . . . 8
2.3 Maven . . . 9
2.4 git . . . 10
2.5 One more useful tool . . . 10

3 Enough terminal to be dangerous 13

3.1 Where am I? . . . 13
3.2 Looking at files . . . 14
3.3 Editing on the Pi . . . 14
3.4 Permissions, and `chmod +x` . . . 15
3.5 `sudo`, and when not to use it . . . 15

3.6 Keys that save you . . . 16
3.7 Environment variables . . . 16

II The Pi **19**

4 Flashing the Pi, headless **21**
4.1 Headless means no screen . . . 21
4.2 Raspberry Pi Imager . . . 21
4.3 The settings screen — the important part . . . 22
4.4 Write it . . . 23
4.5 First boot . . . 24

5 First contact **25**
5.1 Log in . . . 25
5.2 When `brewer.local` does not resolve . . . 26
5.3 Look around . . . 27
5.4 DHCP, and why the address moves . . . 28

6 SSH keys, properly **29**
6.1 What a key pair is . . . 29
6.2 Generate the key . . . 30
6.3 Copy the public key to the Pi . . . 31
6.4 The config file: `ssh brewer` . . . 32
6.5 Turn off password logins . . . 33
6.6 Useful SSH tricks . . . 33

7 Preparing the Pi **35**
7.1 The fast way . . . 35
7.2 Update first . . . 35
7.3 Java and Maven . . . 36

7.4 Enable SPI . . . 36
7.5 Join the hardware groups . . . 37
7.6 Swap, on a 512 MB machine . . . 37
7.7 Reboot and verify . . . 38
7.8 One optional nicety . . . 38

III The Workflow **41**

8 Git and GitHub **43**
8.1 Why bother, on a solo project . . . 43
8.2 Start the repository . . . 44
8.3 Your first commit . . . 44
8.4 GitHub . . . 45
8.5 Push . . . 46
8.6 The loop you will actually use . . . 46

9 The project skeleton **49**
9.1 Copy the template . . . 49
9.2 Build it before you change anything . . . 49
9.3 What is in the folder . . . 50
9.4 Run the toolchain check . . . 50
9.5 Read the test . . . 51
9.6 The Makefile . . . 51
9.7 Point it at your Pi . . . 52
9.8 Commit it . . . 52

10 The deploy loop **55**
10.1 The loop is the thing . . . 55
10.2 Why not just build on the Pi? . . . 56

10.3 One-time: the dependency jars . . . 56
10.4 Every time after that . . . 56
10.5 What the scripts actually do . . . 57
10.6 The rhythm . . . 58

11 An editor that helps **59**
11.1 Why this matters more in Java than in C . . . 59
11.2 VS Code, the pragmatic default . . . 59
11.3 Editing files on the Pi . . . 60
11.4 If you prefer IntelliJ IDEA . . . 61
11.5 Settings worth changing immediately . . . 61

IV The Hardware **63**

12 Wiring the reader **65**
12.1 Power off first . . . 65
12.2 The seven wires . . . 66
12.3 Check it twice . . . 67
12.4 Probe it . . . 67
12.5 If it does not say that . . . 67
12.6 You are done . . . 68

V Appendices **71**

A Cheat sheet **73**
A.1 The daily loop . . . 73
A.2 SSH . . . 73
A.3 Git . . . 74
A.4 Maven . . . 74

A.5 The Pi . . . 75
A.6 Terminal . . . 76
A.7 Pin reference . . . 76
A.8 Values worth memorising . . . 77

B When it does not work **79**
B.1 First, three questions . . . 79
B.2 Cannot reach the Pi . . . 79
B.3 SSH keys still ask for a password . . . 80
B.4 Java and Maven . . . 81
B.5 SPI and the reader . . . 81
B.6 The Pi misbehaves . . . 82
B.7 Git and GitHub . . . 82
B.8 The deploy loop . . . 83
B.9 Still stuck? . . . 83

Before you start

Most project tutorials open with "first, set up your environment" and then spend one paragraph on something that will actually take you two evenings and teach you a dozen things you will use for the rest of your life.

This book is that paragraph, expanded honestly.

What you will have at the end

- A Raspberry Pi you can reach by typing `ssh brewer` — no password, no IP address to remember, from anywhere on your network.
- A Java project that compiles on your laptop and runs on the Pi one second later.
- Your code in git, backed up on GitHub, pushed with one command.
- An NFC reader wired up and answering.

None of that is specific to this project. Change the word "NFC" and you have the setup for any embedded Linux work you will ever do.

Who this is for

Someone who has written some C, can use a text editor, and has never done any of the following: used SSH, generated a key pair, used git beyond `git clone`, run Maven, or set up a headless Linux machine.

If you have done all of those, read Chapter 1 for the hardware choices, skim Appendix A, and go straight to Volume One.

How this book is different from a wiki page

Everything is explained, not just listed. You will not be told to run `ssh-keygen -t ed25519` and left to wonder what happened. You will be told what a key pair *is*, why the private one never moves, and what would actually go wrong if you skipped it.

That matters because setup steps are exactly the ones you cannot debug by reading your own code. When SSH refuses you at midnight, understanding *why* it wants what it wants is the difference between five minutes and an evening.

> **CONCEPT Checkpoints**
>
> Each chapter ends with a **checkpoint**: a command to run, and the output you should see.
> **Do not continue past a failed checkpoint.** Setup problems compound viciously — a half-configured SSH key produces errors three chapters later that look nothing like SSH.

Conventions

Commands are shown with a comment saying *where* to run them, because half of all setup confusion is running the right command on the wrong machine:

```
# on your laptop
ssh brewer

# on the Pi
ls -l /dev/spidev0.0
```

- `$USER`, `martin`, `brewer.local` — replace with your own username and hostname. The book uses `martin` and `brewer` throughout; pick your own in Chapter 4 and stay consistent.
- Red rules mark traps that have actually caught people. Blue explains a concept. Amber is an analogy. Green is a checkpoint.

If you get stuck

Appendix B is indexed by **what you see on screen**, not by topic. Search for your error message there first — almost every setup failure in this book has a known cause and a one-line fix.

Nothing here is hard. Some of it is just unfamiliar.

PART I

Your Machine

CHAPTER 1

What you need

GOAL Buy the right things, once.

CONCEPT The decisions that are expensive to change later.

YOU END WITH A shopping list and no regrets.

1.1 The shopping list

Item	Notes	Approx.
Raspberry Pi Zero 2 W	Any Pi works. See below.	£18
microSD card, 16 GB+	Class 10 / A1. A slow card makes everything feel broken.	£8
USB power supply, 5 V 2.5 A	**Not** a phone charger you had lying around.	£8
MFRC522 reader module	The ubiquitous red breakout board.	£3
MIFARE DESFire EV2/EV3	**Buy two. See the warning.**	£3 ea
Female–female jumper wires	Get short ones, 10 cm. Seriously.	£3
7-inch touchscreen *(optional)*	1024×600, HDMI + USB touch. Volume One, Chapter 12.	£45

You also need a computer (macOS, Linux or Windows), a home network with Wi-Fi, and an SD card reader.

> **TRAP The card matters more than everything else combined**
>
> It must say **DESFire EV2** or **EV3**.
> **Not** MIFARE Classic — broken crypto, completely different protocol. **Not** NTAG or

> Ultralight — no crypto at all, no value files. **Not** "13.56 MHz RFID tag" from a bag of a hundred on an auction site.
> If a listing says "compatible" or does not name the chip, it is not one.
> **Buy at least two.** One to develop against, one spare — because in Volume One you will run a command that changes a key on the card, and a card whose key you have changed and forgotten is scrap. Permanently, by design.

1.2 Which Pi?

Any model with Wi-Fi works. The trade-off:

Model	Good	Annoying
Zero 2 W	Cheap, tiny, fits behind a screen	512 MB RAM; compiling on it is slow
Pi 3 / 4 / 5	Fast, plenty of RAM	Bigger, more power, more money
Zero W (original)	Cheapest	Single core — genuinely painful. Avoid.

This book assumes a **Zero 2 W**, because it is the one that forces you to do things properly: with 512 MB you cannot be careless, and the workflow you learn in Chapter 10 — build on the laptop, run on the Pi — is the right workflow on any hardware.

Everything works unchanged on a Pi 4; you will just wait less.

> **TRAP The power supply is not optional**
>
> A Pi that browns out does not politely stop. It corrupts the SD card, drops the network, and produces failures that look exactly like software bugs.
> If you see random reboots, a rainbow square in the corner, or "undervoltage detected" in `dmesg`, replace the supply before debugging anything else. A phone charger rated "5 V 2 A" often will not hold 5 V under load, and the cable matters as much as the brick.

1.3 Wires: shorter than you think

Get **10 cm female-to-female** jumpers. The 20 cm ones that come in most kits are, at the 1 MHz clock this project uses, genuinely unreliable — and their failure mode is intermittent,

which is the worst kind.

> **ANALOGY Shouting across a room**
>
> A short wire is a conversation at arm's length. A long, unshielded, loosely twisted jumper is shouting across a room with a fan running: most words arrive, some do not, and you cannot predict which.
> Digital signals do not degrade gracefully. A bit that arrives ambiguous is not "slightly wrong" — it is a different number.

You will meet this again in Appendix B: *version register reads a different value each run* is almost always wire length, and almost never code.

1.4 What you do not need

- **A monitor, keyboard or mouse for the Pi.** This book sets it up *headless* — configured before first boot, reached over the network. It is faster, and it is how real deployments work.
- **A soldering iron.** The RC522 usually ships with header pins already attached. If yours came with loose pins in a bag, you do need to solder them, or buy one that is pre-soldered.
- **An Arduino, a level shifter, or a breadboard.** The Pi's GPIO is already 3.3 V, which is what the RC522 wants.

> **CHECKPOINT Before continuing**
>
> You have, physically in front of you: a Pi, an SD card, a power supply, an RC522, jumper wires, and at least one card that says **DESFire EV2** or **EV3** on the packaging.
> If the card is the only thing missing, carry on — you will not need it until Chapter 12.

CHAPTER 2

Your laptop

GOAL Install Java, Maven and git, and prove each one works.

CONCEPT Package managers, and why version numbers matter here.

YOU END WITH Three commands that print version numbers.

Everything in this chapter happens on **your own computer**, not the Pi.

2.1 A package manager first

Installing software by downloading installers from websites is how you end up with three Javas and no idea which one runs. Use a package manager: one command to install, one to update, one to remove.

macOS — Homebrew

```
# on your laptop
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It will ask for your password and print a couple of commands to run afterwards (adding `brew` to your `PATH`) — **actually run them**, then close and reopen your terminal.

Linux (Debian/Ubuntu)

Already installed: `apt`.

Windows

Two options, and this one genuinely matters:

> **TRAP Use WSL2, not native Windows**
>
> Install **WSL2 with Ubuntu** (`wsl -install` in an Administrator PowerShell, then reboot). Do all your work inside that Ubuntu.
> Native Windows will fight you on line endings, file permissions, SSH paths and shell scripts — four separate categories of confusion that have nothing to do with what you are trying to learn. Inside WSL2, every command in this book works exactly as printed. Keep your project files under the Linux home directory (`~/`) and *not* under `/mnt/c/`, or filesystem performance will be dreadful.

2.2 Java 21

```
# macOS
brew install openjdk@21

# Debian / Ubuntu / WSL2
sudo apt update && sudo apt install -y openjdk-21-jdk
```

On macOS, Homebrew does not link the JDK where the system expects it. Run the command it prints, which looks like:

```
sudo ln -sfn /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk \
             /Library/Java/JavaVirtualMachines/openjdk-21.jdk
```

Verify:

```
java -version
```

```
openjdk version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10)
OpenJDK 64-Bit Server VM (build 21.0.11+10, mixed mode)
```

> **CONCEPT Why 21 specifically**
>
> Java 21 is a *Long Term Support* release, and it has one feature this project uses constantly: `HexFormat`, for parsing and printing hexadecimal without pulling in a library.
> Java 17 would mostly work. Java 11 will not compile the code in Volume One. Newer than 21 is fine.
> If `java -version` shows something older, you have two Javas installed and the wrong one is first in your `PATH` — see Appendix B.

2.3 Maven

```
# macOS
brew install maven

# Debian / Ubuntu / WSL2
sudo apt install -y maven
```

```
mvn -v
```

```
Apache Maven 3.9.6
Maven home: /opt/homebrew/Cellar/maven/3.9.6/libexec
Java version: 21.0.11, vendor: Homebrew
```

Check that "Java version" line — Maven has its own idea of which Java to use, and if it says 17 while `java -version` says 21, Maven wins and your build will behave unexpectedly.

> **ANALOGY What Maven actually does**
>
> Think of `pom.xml` as a shopping list plus a recipe.
> The shopping list is your dependencies: "I need Pi4J version 3.0.2". Maven fetches them from a central warehouse and caches them in `~/.m2/`, so the second build is offline and instant.
> The recipe is the build: compile these sources with this Java version, run these tests, put the output there. You do not write build steps; you declare what you want and Maven knows the order.
> The first build downloads a lot and feels slow. That is a one-time cost.

2.4 git

```
# macOS -- may already be present via Xcode tools
brew install git

# Debian / Ubuntu / WSL2
sudo apt install -y git
```

Tell git who you are. This goes into every commit you ever make, so use a real address:

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"

# Name the default branch 'main' -- matches GitHub and this book
git config --global init.defaultBranch main

# Show the config you just set
git config --global --list
```

> **TRAP Your commit email becomes public**
>
> If you push to a public GitHub repository, every commit shows the email you set here, forever, in the git history. It is not editable after the fact without rewriting history. If that bothers you, GitHub can give you a private no-reply address: find it under *Settings* → *Emails* and use that instead. Set it now rather than discovering it in six months.

2.5 One more useful tool

`rsync` copies files efficiently — only the parts that changed. You will use it in Chapter 10 to push builds to the Pi.

```
# macOS: already installed
# Debian / Ubuntu / WSL2
sudo apt install -y rsync

rsync --version | head -1
```

CHECKPOINT All four tools

```
java -version     # 21.x
mvn -v            # 3.9.x, reporting Java 21
git --version     # 2.x
rsync --version   # 3.x
```

All four print a version. None says “command not found”. If Maven reports a different Java version than `java -version` does, fix that now — Appendix B has the two-line solution.

CHAPTER 3

Enough terminal to be dangerous

GOAL The dozen commands this book actually uses.

CONCEPT Paths, permissions, and where you are.

YOU END WITH Confidence that you cannot break anything by accident.

Skip this chapter if `cd`, `ls`, `chmod +x` and `sudo` are already reflexes. Otherwise it is fifteen minutes that will save you hours.

3.1 Where am I?

The shell always has a *current directory*. Almost every confusing error in setup work comes from being somewhere you did not think you were.

```
pwd                 # print working directory -- where am I?
ls                  # list files here
ls -la              # ... including hidden ones, with details
cd ~/projects       # change directory
cd ..               # up one level
cd -                # back to where I just was
```

> **CONCEPT The three special paths**
>
> **~**
> your home directory. On the Pi that is `/home/martin`; on macOS `/Users/martin`.

`.`
here, the current directory. `./script.sh` means "the script in *this* folder" — as opposed to a command found on your `PATH`.

`..`
one level up.

A path starting with `/` is *absolute* — it means the same thing from anywhere. Anything else is *relative* to where you currently are, which is why `pwd` is the first thing to check when a command cannot find a file.

3.2 Looking at files

```
cat file.txt              # dump the whole thing
less file.txt             # page through it (q to quit, / to search)
head -20 file.txt         # first 20 lines
tail -20 file.txt         # last 20 lines
tail -f logfile           # follow it live -- Ctrl-C to stop
grep "error" file.txt     # lines containing "error"
grep -rn "TODO" src/      # search a whole tree, with line numbers
```

`tail -f` is how you watch a program's log while it runs. You will use it in Volume One when the kiosk misbehaves.

3.3 Editing on the Pi

You need one editor that works over SSH. `nano` is the least painful:

```
nano ~/.bashrc
```

Arrow keys move. **Ctrl-O then Enter** saves. **Ctrl-X** exits. Those two shortcuts are all you need, and they are printed at the bottom of the screen — where `^O` means Ctrl-O.

TRAP If you find yourself in `vi`

Some commands (notably `git commit` with no `-m`) open a full-screen editor that ignores everything you type. That is `vim`, and it is modal.

To escape: press **Escape**, then type **`:q!`** and press Enter. That quits without saving.
To make git use nano instead, permanently:

```
git config --global core.editor nano
```

3.4 Permissions, and `chmod +x`

Every file carries permissions: who may read it, write it, and *execute* it.

```
ls -l scripts/deploy.sh
```

```
-rwxr-xr-x  1 martin  staff  1204 Jul 26 14:02 scripts/deploy.sh
 ^^^
 owner may read, write, execute
```

A shell script without the execute bit gives you `Permission denied` even though you can plainly read it. Fix:

```
chmod +x scripts/deploy.sh
```

TRAP Scripts lose the execute bit in transit

Copy a script through a zip file, a Windows filesystem, or some cloud drives and the execute bit is silently dropped.
If a script you *know* is fine says `Permission denied`, run `chmod +x` on it before suspecting anything else. Git does preserve the bit, which is one more reason to use git.

3.5 `sudo`, and when not to use it

`sudo` runs one command as the administrator. Installing packages and editing system configuration need it. **Your own project files never do.**

TRAP Never sudo a build

If you run `sudo mvn compile` even once, the files in `target/` end up owned by root. Your normal user then cannot overwrite them, so the *next* build fails with a permission error — and the obvious-looking fix is more `sudo`, which digs the hole deeper.
If it has already happened:

```
sudo chown -R $USER:$USER ~/midnight-brewer
```

3.6 Keys that save you

Key	Does
Tab	Completes file and command names. Press it constantly. It is also how you check a path exists before pressing Enter.
Up arrow	Previous command. Edit and re-run rather than retyping.
Ctrl-C	Stop the running program.
Ctrl-D	End of input — also logs you out of SSH.
Ctrl-R	Search your command history. Type a few letters of a command you ran last week.
Ctrl-A / Ctrl-E	Jump to start / end of the line.

CONCEPT Tab completion is a correctness tool

It is not just about typing speed. If Tab does not complete a filename, that file *does not exist* where you think it does — and you have found a bug before running anything. Beginners type full paths and get "No such file or directory". Experienced people press Tab, see nothing happen, and immediately run `ls`.

3.7 Environment variables

A variable your shell passes to programs it runs. This book uses one:

```
export PI=martin@brewer.local     # set it for this session
echo $PI                          # read it back
```

`export` only lasts until you close the terminal. To make it permanent, add the line to your shell's startup file:

```
# which shell am I using?
echo $SHELL

# zsh (macOS default)
echo 'export PI=martin@brewer.local' >> ~/.zshrc

# bash (most Linux, WSL2)
echo 'export PI=martin@brewer.local' >> ~/.bashrc
```

Then open a new terminal, or run `source ~/.zshrc` to apply it now.

CHECKPOINT Prove the basics

```
cd ~
pwd                       # prints your home directory
mkdir -p projects/test
cd projects/test
echo "hello" > file.txt
cat file.txt              # prints: hello
ls -l                     # shows file.txt with its permissions
cd ~
rm -rf ~/projects/test    # clean up
```

If all of that worked and you understand what each line did, you know enough terminal for this entire book.

PART II

The Pi

CHAPTER 4

Flashing the Pi, headless

GOAL Write an operating system to the SD card, pre-configured so the Pi joins your Wi-Fi and accepts SSH on first boot.

CONCEPT Headless setup — no monitor, no keyboard, ever.

YOU END WITH A Pi that boots and appears on your network.

4.1 Headless means no screen

You are never going to plug a monitor into this Pi. Instead you configure it *before* first boot, and from then on you reach it over the network.

> **ANALOGY Setting up a phone before you unbox it**
>
> Imagine you could type your Wi-Fi password and set your account name into a phone *before* it first powered on — so that the moment it boots it is already on your network and already yours.
> That is what the Raspberry Pi Imager's settings do. The alternative is finding a spare HDMI cable, a keyboard, and somewhere to put them.

4.2 Raspberry Pi Imager

Download it from `raspberrypi.com/software` and install it. It runs on macOS, Windows and Linux.

Insert your SD card, open Imager, and make three choices:

Device
Your Pi model — e.g. *Raspberry Pi Zero 2 W*.

Operating System
Raspberry Pi OS Lite (64-bit), under *Raspberry Pi OS (other)*.

Storage
Your SD card. **Read this twice.**

> **TRAP Imager will happily erase the wrong disk**
>
> It writes to whatever you select and does not undo. Before clicking, confirm the *size* matches your SD card — a 32 GB entry is your card, a 1 TB entry is your laptop. Unplug other external drives while you do this. People have lost backups here.

> **CONCEPT Why "Lite", and why 64-bit**
>
> **Lite** means no desktop environment. You do not need one — you will use SSH — and on a 512 MB Zero 2 W a desktop consumes memory your builds need.
> **64-bit** matters because the Java 21 packages are better supported there, and the JVM performs measurably better.
> (Volume One's kiosk chapter does need a graphical desktop. You will install just the pieces required at that point, rather than carrying a full desktop through every chapter before it.)

4.3 The settings screen — the important part

Before writing, click the **gear icon** (or answer *"Would you like to apply OS customisation settings?"* with **Edit Settings**).

General tab

Field	Set it to
Set hostname	`brewer` This is how you will address the Pi: `brewer.local`. Pick something short you will not mistype.
Set username and password	Username `martin` (or yours), and a real password. You will only type this password a handful of times before Chapter 6 replaces it with a key.
Configure wireless LAN	Your Wi-Fi name (SSID) and password, and your **Wireless LAN country** — required, or Wi-Fi stays off.
Set locale settings	Your timezone and keyboard layout.

> **TRAP The Zero 2 W is 2.4 GHz only**
>
> It cannot see a 5 GHz network. If your router presents both bands under different names, enter the **2.4 GHz** one.
> If both bands share one name, it usually still works — but "the Pi never appears on the network" with correct-looking settings is very often this.

Services tab

Enable SSH, and choose **Use password authentication** for now. You will switch to keys in Chapter 6.

> **CONCEPT What SSH is**
>
> *Secure Shell*: a program that gives you a terminal on another machine over the network, with everything encrypted in transit.
> When you type `ssh martin@brewer.local`, you get a shell prompt on the Pi. Commands you type run *there*. It is the single most important tool in embedded Linux work, and Chapter 6 is entirely about making it pleasant.

4.4 Write it

Click **Save**, then **Write**, and confirm. It takes a few minutes — it writes, then verifies.

When it finishes, macOS may complain that "the disk you inserted was not readable". **That is**

normal: the card now has a Linux filesystem your laptop cannot read. Eject it and ignore the warning.

4.5 First boot

1. Put the SD card in the Pi.
2. On a Zero 2 W, plug power into the port marked **PWR IN** — the one nearer the corner. The other is USB data and will not power it.
3. Wait. **First boot takes two to three minutes** — it resizes the filesystem and reboots itself once.

The green LED tells you what is happening: irregular flickering is disk activity, which is good. A steady green light with no flicker for several minutes usually means it has not booted.

> **CHECKPOINT The Pi is on the network**
>
> From your laptop:
>
> ```
> ping brewer.local
> ```
>
> ```
> PING brewer.local (192.168.1.42): 56 data bytes
> 64 bytes from 192.168.1.42: icmp_seq=0 ttl=64 time=4.331 ms
> ```
>
> Ctrl-C to stop. Note the IP address — you may need it in the next chapter.
> **If it does not answer**, wait a full three minutes and try again before troubleshooting. If it still fails, go to Chapter 5, which is entirely about finding a Pi that will not answer to its name.

CHAPTER 5

First contact

GOAL Find the Pi and log into it.

CONCEPT Hostnames, mDNS, DHCP — how machines find each other.

YOU END WITH A shell prompt that says `martin@brewer`.

5.1 Log in

```
# on your laptop
ssh martin@brewer.local
```

The first time, you get this:

```
The authenticity of host 'brewer.local (192.168.1.42)' can't be established.
ED25519 key fingerprint is SHA256:xK3f...9dQ.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type **`yes`** and press Enter. Then your password (it shows nothing as you type — that is deliberate, not a broken keyboard).

```
martin@brewer:~ $
```

That prompt means you are on the Pi. Everything you type now runs there.

> **CONCEPT What that fingerprint question is**
>
> Every SSH server has its own identity key. The first time you connect, your computer

> has no way to know whether this is really your Pi or something pretending to be it, so it shows you the fingerprint and records your answer in `~/.ssh/known_hosts`.
> From then on it checks silently. If the fingerprint ever *changes*, SSH refuses to connect and warns loudly — because that is what an impersonation attack looks like.
> On a home network, the realistic cause of that warning is that you reflashed the Pi (new key) or your router handed the address to a different device. See Appendix B for the one-line fix — and note that the fix is to remove the *specific stale entry*, never to disable checking.

5.2 When `brewer.local` does not resolve

`.local` names use **mDNS** — machines shouting "I am brewer" on the local network, with no central server. It works out of the box on macOS and most Linux, and on Windows 10+.

When it fails, work down this list.

5.2.1 1. Is it a name problem or a network problem?

Find the Pi's IP from your router's admin page (usually `192.168.1.1` in a browser; look for "connected devices" or "DHCP leases"). Then:

```
ssh martin@192.168.1.42
```

If **that works**, the Pi is fine and only the name lookup is broken — fix it with the `~/.ssh/config` entry in Chapter 6, which lets you type `ssh brewer` regardless.

If **that fails too**, the Pi is not on the network. Continue.

5.2.2 2. Scan the network

```
# macOS / Linux -- find your own address first
ipconfig getifaddr en0        # macOS
hostname -I                   # Linux

# then scan that subnet for open SSH ports
nmap -p 22 --open 192.168.1.0/24
```

Install `nmap` with `brew install nmap` or `sudo apt install nmap`. Any device answering on

port 22 is running SSH; if you only have one Pi, that is it.

5.2.3 3. Still nothing

In order of likelihood:

Cause	Check
Wi-Fi credentials wrong	Re-flash the card with Imager, retyping the SSID and password carefully. Both are case-sensitive.
5 GHz network selected	A Zero 2 W cannot see 5 GHz. Use the 2.4 GHz SSID.
Wireless country not set	Imager's *Wireless LAN country* is required; without it the radio stays off.
SSH not enabled	Services tab in Imager. Easy to miss.
Power	Undervoltage. Try a different supply *and* cable.
Card not written	Imager reported an error, or you ejected early.

TRAP Re-flashing is faster than diagnosing

If the Pi has never appeared, do not spend an hour on it. Re-flash with Imager, type the Wi-Fi details slowly, and try again. It takes five minutes, and it fixes the great majority of never-appeared cases.

Diagnosis is worth it once the Pi has worked at least once — then you know the hardware is good and something specific changed.

5.3 Look around

You are logged in. Confirm what you are on:

```
# on the Pi
hostname                              # brewer
whoami                                # martin
cat /proc/device-tree/model; echo     # Raspberry Pi Zero 2 W Rev 1.0
free -h                               # memory: total / used / free
df -h /                               # disk space
uname -m                              # aarch64  <- 64-bit, as intended
ip addr show wlan0 | grep 'inet '     # your IP address
```

Two things worth noting now:

`free -h` on a Zero 2 W shows about 420 MB available. That is why Chapter 7 adds swap and Chapter 10 builds on your laptop.

`uname -m` must say `aarch64`. If it says `armv7l` you flashed the 32-bit image — workable, but re-flash with the 64-bit one now rather than later.

To leave:

```
exit          # or Ctrl-D
```

5.4 DHCP, and why the address moves

Your router hands out addresses on a lease. When the lease expires, or the Pi reboots, or another device claims it first, the Pi can get a *different* address.

That is why this book uses `brewer.local` rather than a number. If mDNS is unreliable on your network, the durable fix is a **static DHCP lease** (also called an address reservation) in your router: find the Pi in the device list and tick "always give this device this address". Takes a minute, and it is worth doing before you start filming or demoing anything.

> **CHECKPOINT You can log in**
>
> ```
> ssh martin@brewer.local
> hostname
> exit
> ```
>
> You reached a prompt, it printed `brewer`, and you got back out. You typed a password to do it — that is what the next chapter fixes.

CHAPTER 6

SSH keys, properly

GOAL Replace the password with a key, and make `ssh brewer` work from anywhere on your network.

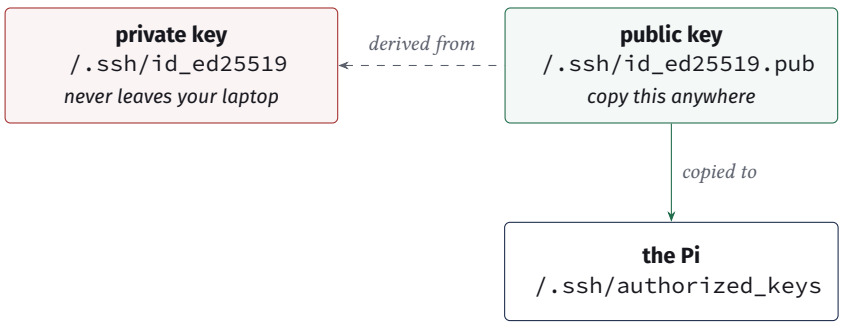
CONCEPT Public-key cryptography, from the user's side.

YOU END WITH A one-word login, and no passwords in your workflow.

This is the chapter that changes how the rest of the project feels. Typing a password every time you deploy is not merely tedious — it is the reason people stop deploying often, and deploying often is the entire point of Chapter 10.

6.1 What a key pair is

You generate two mathematically linked files:



> **ANALOGY A padlock you hand out freely**
>
> The **public key** is an open padlock. You can make unlimited copies and leave them anywhere — on the Pi, on GitHub, on a colleague's server. A padlock is not a secret.
> The **private key** is the only key that opens those padlocks. It stays in one place and is never copied, never emailed, never pasted into a chat.
> When you connect, the Pi holds up your padlock and says "prove you can open this". Your laptop proves it *without ever sending the key* — and that is the same trick, mathematically, that the DESFire card uses in Volume One, Chapter 10. You are about to use challenge–response before you implement it.

6.2 Generate the key

```
# on your laptop
ssh-keygen -t ed25519 -C "martin@laptop"
```

Three questions:

File to save the key
Press Enter for the default, `~/.ssh/id_ed25519`. Only change it if you already have a key you want to keep.

Passphrase
Press Enter twice for none, *or* set one — see the box below.

Same again
Confirm.

```
Your identification has been saved in /Users/martin/.ssh/id_ed25519
Your public key has been saved in /Users/martin/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:xK3f...9dQ martin@laptop
```

> **CONCEPT `ed25519`, and passphrases**
>
> **Why ed25519 and not RSA?** It is a modern elliptic-curve algorithm: shorter keys, faster, and no key-size decision to get wrong. Use RSA only if you must talk to something ancient.

> **Should you set a passphrase?** It encrypts the private key on disk, so a stolen laptop does not hand over your servers. The cost is typing it — but your operating system's keychain remembers it after the first use, so in practice you type it once per reboot. For a home Pi with nothing valuable on it, no passphrase is a defensible choice. For the key you will put on GitHub in Chapter 8, set one.

6.3 Copy the public key to the Pi

```
ssh-copy-id martin@brewer.local
```

It asks for your Pi password one last time, then:

```
Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'martin@brewer.local'"
```

Test it:

```
ssh martin@brewer.local
```

No password. That is the whole point.

If `ssh-copy-id` is missing (some macOS setups)

```
brew install ssh-copy-id

# or do it by hand -- this is exactly what the tool does:
cat ~/.ssh/id_ed25519.pub | ssh martin@brewer.local \
  "mkdir -p ~/.ssh && chmod 700 ~/.ssh && \
   cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

> **TRAP SSH refuses keys if the permissions are loose**
>
> `~/.ssh` must be `700` and `authorized_keys` must be `600`. If they are group- or world-writable, the SSH server silently ignores the key and falls back to asking for a password. Silently is the operative word: no error, no log line you would notice, just a password

prompt that should not be there. If keys "do not work" and you have checked everything else, check permissions:

```
# on the Pi
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

6.4 The config file: `ssh brewer`

Typing `martin@brewer.local` forever is unnecessary. On your laptop, create `~/.ssh/config`:

```
nano ~/.ssh/config
```

```
Host brewer
    HostName brewer.local
    User martin
    IdentityFile ~/.ssh/id_ed25519

    # Keep the connection alive through Wi-Fi hiccups instead of
    # dropping you mid-deploy.
    ServerAliveInterval 30
    ServerAliveCountMax 6
```

```
chmod 600 ~/.ssh/config
ssh brewer
```

That is it. One word.

CONCEPT Why this is more than a shortcut

`~/.ssh/config` is read by *everything* that uses SSH — `scp`, `rsync`, `git`, and the deploy scripts in Chapter 10. Define `brewer` once and it works everywhere:

```
scp file.txt brewer:~
rsync -a target/classes/ brewer:midnight-brewer/target/classes/
```

It also means that when your Pi's address changes, you edit *one line* instead of hunting

through scripts.
If mDNS is unreliable on your network, put the IP in `HostName` instead — everything else keeps working unchanged. That indirection is the real value.

6.5 Turn off password logins

Now that keys work, close the door behind you.

```
# on the Pi
sudo nano /etc/ssh/sshd_config
```

Find and set these three (they may exist commented out with a `#` — remove it):

```
PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
```

Save (Ctrl-O, Enter), exit (Ctrl-X), then:

```
sudo systemctl restart ssh
```

TRAP Keep your current session open

Do not close your SSH window until you have proved a new one works.
Open a *second* terminal and run `ssh brewer`. If it lets you in, you are safe. If it does not, you still have the first window to undo the change.
If you close everything and locked yourself out, the recovery is physical: power off, put the SD card in your laptop, and edit the config on the boot partition — or just re-flash. Not fatal, but a genuinely annoying half hour.

Why bother, on a home network? Because a Pi with password login and a weak password is exactly what automated scanners look for, and because the habit matters more than this one machine.

6.6 Useful SSH tricks

```
# run one command without opening a shell
ssh brewer 'uptime'

# copy a file over
scp notes.txt brewer:~

# copy a whole folder back
scp -r brewer:~/logs ./logs

# see what SSH is actually doing, when it misbehaves
ssh -v brewer
```

`ssh -v` is the diagnostic. It prints every step — which key it offered, what the server said — and it is how you find out that the server ignored your key because of file permissions.

CHECKPOINT Keys work, passwords are off

```
# from your laptop, in a NEW terminal
ssh brewer 'hostname && whoami'
```

```
brewer
martin
```

No password prompt, one word to connect, and the command ran remotely. You are now set up better than most working developers.

CHAPTER 7

Preparing the Pi

GOAL Install Java and Maven, enable SPI, join the hardware groups, add swap.

CONCEPT Why a 512 MB machine needs care.

YOU END WITH A Pi that can build and run this project.

7.1 The fast way

The starter pack ships a script that does everything in this chapter. Read the chapter anyway — when something fails later you want to know what was done.

```
# on your laptop, from the template folder
scp scripts/pi-bootstrap.sh brewer:~
ssh brewer 'bash pi-bootstrap.sh'
```

Then reboot and skip to the checkpoint:

```
ssh brewer 'sudo reboot'
```

The rest of this chapter is the same work, done by hand.

7.2 Update first

```
# on the Pi
sudo apt update
sudo apt full-upgrade -y
```

On a Zero 2 W this takes a while on a fresh image. Let it finish.

> **CONCEPT `update` versus `upgrade`**
>
> `apt update` refreshes the *catalogue* — what versions exist. It installs nothing.
> `apt upgrade` installs newer versions of what you already have. `full-upgrade` additionally allows removing packages when a dependency requires it, which is what you want on a fresh system.
> Running `upgrade` without `update` first is the most common apt mistake: it dutifully upgrades to the newest versions it knows about, which may be months old.

7.3 Java and Maven

```
sudo apt install -y openjdk-21-jdk maven git rsync
java -version
mvn -v
```

```
openjdk version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1-deb13u2-Debian)
```

Maven on the Pi is used rarely — Chapter 10 builds on your laptop — but it is needed once, to download the dependency jars.

7.4 Enable SPI

SPI is the bus the NFC reader speaks. It is off by default.

```
sudo raspi-config
```

Navigate: **3 Interface Options** → **I4 SPI** → **Yes** → **Ok** → **Finish**.

Or without the menu:

```
sudo raspi-config nonint do_spi 0      # 0 means ENABLE, confusingly
```

7.5 Join the hardware groups

Linux does not let arbitrary programs poke hardware. Access is granted by group membership.

```
sudo usermod -aG spi,gpio $USER
```

> **TRAP Group changes need a fresh login**
>
> Your current session still has your *old* groups. Adding yourself to a group does nothing until you log out and back in — and over SSH that means disconnecting, not just opening a new shell.
> This produces one of the most confusing errors in the whole setup: you ran the command, you can see the group in `/etc/group`, and the program still says `Permission denied`.
>
> ```
> exit
> ssh brewer
> groups          # must now list: spi gpio
> ```

7.6 Swap, on a 512 MB machine

A Zero 2 W has 512 MB of RAM. A JVM plus Maven will exceed that, and when Linux runs out of memory the kernel starts killing processes — frequently `sshd`, which locks you out until you power-cycle it.

```
free -h                    # check what you have

sudo dphys-swapfile swapoff
sudo nano /etc/dphys-swapfile
```

Change `CONF_SWAPSIZE=100` to `CONF_SWAPSIZE=1024`, then:

```
sudo dphys-swapfile setup
sudo dphys-swapfile swapon
free -h                    # Swap: should now show ~1.0Gi
```

> **CONCEPT Swap is a safety net, not speed**
>
> Swap is disk space used as overflow memory. Disk is thousands of times slower than RAM, so a machine actively swapping is a machine crawling.
> That is fine — the goal is not performance, it is *not dying*. Without swap, running out of memory kills your SSH session. With swap, it merely gets slow, and you can still type. On an SD card, heavy swapping does wear the card. Which is one more argument for building on your laptop.

7.7 Reboot and verify

```
sudo reboot
```

Wait thirty seconds, reconnect, and check everything:

```
ssh brewer

ls -l /dev/spidev0.0        # the SPI device must exist
groups                      # must include spi and gpio
java -version               # 21.x
free -h                     # swap around 1 GB
uname -m                    # aarch64
```

```
crw-rw---- 1 root spi 153, 0 Jul 26 14:02 /dev/spidev0.0
martin adm dialout cdrom sudo audio video plugdev games users input
       render netdev spi gpio lpadmin
```

Read that first line: the device is owned by group `spi`, and the group has read/write (`rw`). That is exactly why you needed the group membership.

7.8 One optional nicety

Give yourself a clock and a tidy history:

```
# on the Pi
cat >> ~/.bashrc <<'EOF'

# timestamps in history -- surprisingly useful when debugging
```

```
export HISTTIMEFORMAT="%F %T  "
export HISTSIZE=5000
export HISTCONTROL=ignoredups

# shortcuts
alias ll='ls -alh'
alias ..='cd ..'
EOF

source ~/.bashrc
```

CHECKPOINT The Pi is ready

```
ssh brewer 'ls -l /dev/spidev0.0 && groups | tr " " "\n" | grep -E "^(spi|gpio)$"
   ↪  && java -version'
```

You should see the SPI device, both groups, and Java 21.
If `/dev/spidev0.0` does not exist, SPI is not enabled — rerun `raspi-config` and *reboot*.
If the groups are missing, log out and back in.

PART III

The Workflow

CHAPTER 8

Git and GitHub

GOAL Put your work under version control and back it up.

CONCEPT Commits as save points; SSH keys, a second time.

YOU END WITH `git push` working with no password.

8.1 Why bother, on a solo project

Three reasons, and only the first is the obvious one.

Backup.
SD cards die. Laptops get spilled on.

Undo.
Not `Ctrl-Z` undo — *"the reader worked on Tuesday and does not now"* undo. Git can show you exactly what changed between Tuesday and today.

A record of your reasoning.
In Volume One each chapter ends with a commit. Six months later, `git log` is a narrative of what you built and in what order — which is worth more than you expect when you come back to it, or film it.

> **ANALOGY Save points, not autosave**
>
> A word processor autosaves continuously; you cannot go back to "before I rewrote that section".
> Git is save points in a game. *You* decide when a moment is worth marking, and you name it. "M2: driver reads the version register" is a save point you can return to, compare

against, or branch from.
The skill is choosing when to save: after something works, before something risky.

8.2 Start the repository

You will do this inside the project folder from Chapter 9. If you have not created it yet, come back after that chapter.

```
# on your laptop, inside the project folder
git init
git status
```

`git status` is the command you will run most. It answers: what have I changed, and what is staged?

8.3 Your first commit

```
git add .                  # stage everything not ignored
git status                 # confirm what is staged -- always look
git commit -m "chore: project skeleton from the starter pack"
git log --oneline
```

CONCEPT Why two steps, `add` then `commit`

`add` stages changes; `commit` records what is staged.
The separation exists so you can commit *part* of your work. You fixed a bug and also renamed a variable; those are two different stories, and staging lets you tell them separately.
At first, `git add .` then `git commit` is fine. Later, `git add -p` steps through each change and asks whether to include it — and that is when the two-step design starts earning its keep.

Commit messages that help later

A convention worth adopting, because it makes `git log` scannable:

```
feat:  a new capability          feat: read the card UID
fix:   a bug fixed               fix: skip the 0x88 cascade tag
chore: housekeeping              chore: add .gitignore
docs:  documentation             docs: explain the FIFO wait loop
test:  tests only                test: RFC 4493 vectors for CMAC
```

Write the message as the completion of *"this commit will…"* — present tense, imperative. `fix: skip the cascade tag`, not `fixed the cascade tag`.

> **TRAP What must never be committed**
>
> `.gitignore` in the starter pack already excludes `target/` and common secret filenames. Two habits on top of that:
>
> - **Never commit a private key.** `id_ed25519` stays in `~/.ssh/`, never in a project.
> - **Never commit a password.** Not in a config file, not in a comment, not "temporarily".
>
> Git history is permanent. Deleting a secret in a later commit does *not* remove it — it is still there in the history, and if you pushed it publicly, assume it is compromised and rotate it.

8.4 GitHub

Create an account at `github.com` if you do not have one, then a new repository: **New** → name it `midnight-brewer` → **Private** → and **do not** tick any of "add a README / .gitignore / licence" (you already have files; those options create a conflicting first commit).

8.4.1 An SSH key for GitHub

You can push over HTTPS with a token, but SSH is less friction and you already know how it works.

```
# on your laptop -- reuse the key you already made
cat ~/.ssh/id_ed25519.pub
```

Copy the whole line (it starts `ssh-ed25519` and ends with your comment). In GitHub: **Settings** → **SSH and GPG keys** → **New SSH key**, paste, save.

Test:

```
ssh -T git@github.com
```

```
Hi evilcomputer12! You've successfully authenticated,
but GitHub does not provide shell access.
```

That message is success — GitHub is not supposed to give you a shell.

> **CONCEPT The same padlock, a second lock**
>
> This is the identical key pair from Chapter 6. You handed a copy of the public half to your Pi; now you have handed another copy to GitHub.
> One private key, many places that recognise it. That is the whole model, and it is why the private key never moving matters: it is the single thing whose compromise would unlock everything at once.

8.5 Push

```
git remote add origin git@github.com:YOURNAME/midnight-brewer.git
git branch -M main
git push -u origin main
```

```
Enumerating objects: 14, done.
Writing objects: 100% (14/14), 3.21 KiB | 3.21 MiB/s, done.
To github.com:YOURNAME/midnight-brewer.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Refresh the GitHub page — your files are there.

The `-u` sets `origin/main` as the default, so from now on it is just:

```
git push
```

8.6 The loop you will actually use

```
git status                    # what changed?
git diff                      # show me the actual changes
git add -A                    # stage everything
git commit -m "feat: ..."     # save point
git push                      # back it up
```

And two that save you:

```
git log --oneline -10         # last ten save points
git diff HEAD~1               # what changed in the last commit
git checkout -- file.java     # throw away my changes to this file
```

TRAP `git checkout --` is destructive

That last command discards your uncommitted changes to that file, *permanently*. There is no undo, because the changes were never recorded.
That is precisely why you commit often: committed work can always be recovered; uncommitted work is one careless command from gone.

CHECKPOINT Your code is backed up

```
git log --oneline
git push
```

At least one commit exists, `push` reports *Everything up-to-date*, and you can see your files on github.com.

CHAPTER 9

The project skeleton

GOAL Copy the template, build it, run its tests — all on your laptop.

CONCEPT What Maven's folder layout means, and why it is not negotiable.

YOU END WITH `Tests run: 3, Failures: 0`.

9.1 Copy the template

```
# on your laptop
cp -r starter-pack/template ~/midnight-brewer
cd ~/midnight-brewer
ls -la
```

9.2 Build it before you change anything

```
mvn test
```

The first run downloads dependencies and takes a minute or two. Afterwards it is seconds.

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

> **TRAP Read the number, not the word**
>
> `BUILD SUCCESS` with *Tests run: 0* is also a success, and it means your tests are not being

found at all.
This is the single most common silent failure in Java projects: a misplaced test folder, a class not ending in `Test`, an old Surefire version that cannot see JUnit 5. Everything looks green and nothing is verified.
Always check the count. Here it must say 3.

9.3 What is in the folder

```
midnight-brewer/
   pom.xml                 what to build, which libraries
   Makefile                the commands you type
   .gitignore              keeps target/ out of git
   README.md               how to use it
   scripts/
      pi-bootstrap.sh      run once, on the Pi
      deploy.sh            build here, copy there
      run.sh               run a class on the Pi
   src/main/java/...       your program
   src/test/java/...       your tests
   target/                 build output – never edited, never committed
```

CONCEPT Why `src/main/java` and not something shorter

Maven follows *convention over configuration*: it knows where sources live, so you never write it down.
Put a file in the wrong place and it is silently not compiled — no error, it simply does not exist as far as the build is concerned. That is the trade: total consistency across every Maven project in the world, at the cost of paths being fixed.
The package `com.midnightbrewer.diag` must map to the folder `src/main/java/com/midnightbrewer/diag/`. Not similar — identical.

9.4 Run the toolchain check

```
mvn -q compile
java -cp target/classes com.midnightbrewer.ToolchainCheck
```

```
=== toolchain check ===
java.version    = 21.0.11
os.name         = Mac OS X
os.arch         = aarch64
major version   = 21
sanity          = 0x92 (146)

  -> toolchain OK. Nothing above needed hardware.
```

CONCEPT What `-cp` means

`-cp` is the *classpath*: where Java looks for compiled classes.
`target/classes` is where Maven put your compiled code. Later you will use `-cp "target/classes:target/dependency/*"` to include the library jars as well — the colon separates entries, and the `*` means "every jar in that folder".
Most "`ClassNotFoundException`" and "`NoClassDefFoundError`" problems are a classpath missing an entry, not a missing file.

9.5 Read the test

Open `src/test/java/com/midnightbrewer/ToolchainCheckTest.java`. One test is worth understanding now, because it encodes the mistake you are most likely to make in Volume One:

```
@Test
void signedByteMaskingWorks() {
    byte b = (byte) 0x92;

    assertEquals(-110, b, "a signed byte holding 0x92 reads as -110");
    assertEquals(146, b & 0xFF, "masked with 0xFF it reads as 146");
}
```

Java's `byte` is signed and there is no `unsigned` keyword. `0x92` has its top bit set, so read as a number it is −110. The idiom `& 0xFF` promotes to `int` and strips the sign extension.

You will forget this at least once. Everyone does.

9.6 The Makefile

```
make            # lists every target
```

```
Midnight Brewer -- available targets:

  build        compile locally
  test         run the tests locally
  deploy       build + copy classes to the Pi
  run          run MAIN on the Pi
  probe        run the RC522 wiring probe on the Pi
  pi-setup     copy + run the one-time Pi bootstrap script
  ssh          open a shell on the Pi
```

> **CONCEPT Why a Makefile and not just typing the commands**
>
> Because you will type these hundreds of times, and because six months from now `make deploy` still works when you have forgotten the rsync flags.
> A Makefile is executable documentation. The commands are recorded in the repository, they are the same for everyone, and `make` with no arguments tells you what is available — which is more than most projects manage.

9.7 Point it at your Pi

```
export PI=martin@brewer
```

Add that to `~/.zshrc` or `~/.bashrc` so it survives a new terminal. It uses the `brewer` alias from your `~/.ssh/config`.

9.8 Commit it

Now do Chapter 8 if you have not:

```
git init
git add .
git commit -m "chore: project skeleton from the starter pack"
```

CHECKPOINT The project builds

```
cd ~/midnight-brewer
mvn test
make
```

`Tests run: 3, Failures: 0`, and `make` lists its targets.
No hardware has been involved yet. That is the point: when something breaks in Volume One, you already know the toolchain is not the problem.

CHAPTER 10

The deploy loop

GOAL Build on your laptop, run on the Pi, in about five seconds.

CONCEPT Why the feedback loop's length decides how you work.

YOU END WITH `make deploy && make run` as one reflex.

10.1 The loop is the thing

How long it takes to see the effect of a change determines how you behave.

Loop	How you end up working
5 s	Try it. Wrong? Change one thing, try again. You experiment.
40 s	You batch up three changes to avoid the wait — and when it fails you no longer know which one did it.
3 min	You stop running it. You read the code and *reason*, which is far more error-prone than looking.

The middle row is the dangerous one, because it feels productive.

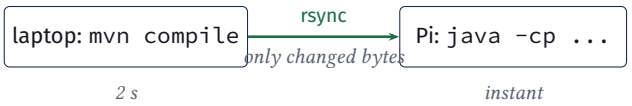
> **ANALOGY Tasting while you cook**
>
> A cook tastes constantly — a pinch of salt, taste, another pinch. Small corrections, immediate feedback.
> Now imagine each taste took three minutes. You would salt once, hopefully, and find out at the table. That is what a slow build does to programming: it turns tasting into guessing.

10.2 Why not just build on the Pi?

You could. `git push` from the laptop, `git pull` on the Pi, `mvn compile`. It works, and on a Zero 2 W it takes roughly forty seconds.

The trick is that **Java bytecode is architecture-independent**. A `.class` file compiled on your laptop — x86 or Apple Silicon, does not matter — runs unchanged on the Pi's ARM chip. The JVM is the portability layer.

So: compile on the fast machine, copy the output to the slow one.



10.3 One-time: the dependency jars

Your code needs Pi4J and SLF4J at runtime. Those jars change only when `pom.xml` changes, so copy them once:

```
# on your laptop, in the project
make deploy-full
```

That builds, resolves the dependency jars into `target/dependency/`, and copies both to the Pi.

10.4 Every time after that

```
make deploy
make run
```

```
==> building locally
==> copying classes to martin@brewer:~/midnight-brewer
==> done
=== toolchain check ===
java.version    = 21.0.11
os.arch         = aarch64            <- note: you are on the Pi now
major version   = 21
  -> toolchain OK.
```

That `os.arch = aarch64` is the proof: you compiled on your laptop and the result ran on ARM.

To run something else:

```
make run MAIN=com.midnightbrewer.diag.SpiProbe
# or equivalently
./scripts/run.sh com.midnightbrewer.diag.SpiProbe
```

10.5 What the scripts actually do

Worth reading once, so they are not magic.

`deploy.sh`

```
mvn -q -DskipTests compile
rsync -a --delete target/classes/ "$PI:midnight-brewer/target/classes/"
```

`-DskipTests`
tests already ran on your laptop; running them again on the Pi would be slow and pointless.

`rsync -a`
archive mode: recursive, preserves timestamps and permissions.

`--delete`
remove files on the Pi that no longer exist locally. Without it, a class you renamed lingers over there — and stale classes produce genuinely baffling behaviour.

the trailing slashes
`target/classes/` means *the contents of*. Without the slash rsync would create `classes/classes/`. This bites everyone once.

`run.sh`

```
ssh -t "$PI" "cd ~/midnight-brewer && java -cp 'target/classes:target/dependency/*'
   ↪ $MAIN"
```

`-t` forces a terminal, so **Ctrl-C reaches the program on the Pi** instead of only killing your local ssh client and leaving the program running over there. Without it you accumulate

orphaned processes holding the SPI device, and the next run fails with a resource-busy error you will not understand.

> **TRAP Stale classes**
>
> If you change a file and behaviour does not change, suspect stale output before suspecting anything else.
>
> ```
> mvn clean compile && make deploy-full
> ```
>
> The symptom is unmistakable once you know it: the code in front of you *cannot* produce the output you are seeing.

10.6 The rhythm

```
# edit code in your editor
mvn test                 # fast, local, no hardware
make deploy && make run  # on the real thing
```

Most of your loop is the middle line. The bottom line is for when you need actual hardware — and by Volume One, Chapter 1, you will have arranged for that to be as rarely as possible.

> **CHECKPOINT The loop works**
>
> ```
> make deploy-full
> make run
> ```
>
> You see the toolchain check, printed by the Pi, reporting `aarch64`.
> Now change one line in `ToolchainCheck.java` — add a `System.out.println("hello from the loop");` — and run:
>
> ```
> make deploy && make run
> ```
>
> Your new line appears. Time it: it should be around five seconds. That number is what the rest of the project will feel like.

CHAPTER 11

An editor that helps

GOAL Set up an editor that understands Java, so mistakes surface as you type.

CONCEPT What a language server does for you.

YOU END WITH Autocomplete, red squiggles, and jump-to-definition.

11.1 Why this matters more in Java than in C

Java is verbose, and a good editor absorbs most of that verbosity: it writes the imports, completes the method names, and underlines the type error before you compile.

Without one, you will spend a real fraction of Volume One fixing errors an editor would have shown you as you typed. With one, Java stops feeling wordy.

11.2 VS Code, the pragmatic default

Free, works everywhere, and its remote-SSH support is genuinely excellent.

1. Install from `code.visualstudio.com`.
2. Open the Extensions panel (the four-squares icon) and install **Extension Pack for Java** by Microsoft. That single pack pulls in the language server, debugger, Maven and test support.
3. Open your project: **File** → **Open Folder** → `~/midnight-brewer`.

The first time it opens a Java project it spends a minute indexing. Let it.

What you get

Feature	Why it matters here
Red squiggles	Type errors before you run `mvn`.
Ctrl-Space	Completes method names — and shows what a class offers, which is how you learn an unfamiliar API.
F12 / Ctrl-click	Jump to a definition. Invaluable when reading Pi4J.
Ctrl-Shift-O	Auto-add missing imports.
Run / Debug lens	A clickable *Run* above every `main` and every `@Test`.
Breakpoints	Stop mid-execution and inspect variables. Debugging the crypto in Volume One will feel very different with these.

> **CONCEPT What a "language server" is**
>
> The intelligence is not in the editor. It is a separate program — for Java, Eclipse JDT — that parses your project, understands the types, and answers questions like *"what methods does this variable have?"*
> The editor just displays those answers. The same server backs VS Code, Vim and Emacs, which is why the experience is comparable across all three now.
> It also explains a common confusion: the language server has its own view of your project, so occasionally it disagrees with Maven. When errors look impossible, **Ctrl-Shift-P → "Java: Clean Java Language Server Workspace"** resolves most of it.

11.3 Editing files on the Pi

Two approaches, and the first is better than it sounds.

11.3.1 Edit locally, deploy (recommended)

Your code lives on your laptop, git-tracked, in a proper editor. The Pi only ever receives compiled output. That is what Chapter 10 set up, and it is the right default: the Pi holds no state you would cry about losing.

11.3.2 Remote-SSH, for config files

Sometimes you need to edit something that only exists on the Pi — `/etc/ssh/sshd_config`, a systemd unit. Install the **Remote — SSH** extension, then:

Ctrl-Shift-P → *Remote-SSH: Connect to Host* → `brewer`

It reuses your `~/.ssh/config`, so the alias you made in Chapter 6 appears in the list. You now have a full editor running against the Pi's filesystem.

> **TRAP Remote-SSH on a Zero 2 W**
>
> VS Code installs a server component on the Pi, and on 512 MB it is heavy — sometimes heavy enough to trigger the out-of-memory killer, which takes `sshd` with it.
> Use it for quick config edits, not as your main workspace. If the Pi becomes unresponsive after connecting, that is what happened; power-cycle and prefer `nano` over SSH for small edits.

11.4 If you prefer IntelliJ IDEA

The Community Edition is free and is, for Java specifically, better than VS Code — stronger refactoring, better test integration, a nicer debugger.

Open the project by selecting `pom.xml` and choosing *Open as Project*. It has no remote-SSH equivalent, but with the deploy loop you barely need one.

Either is fine. Do not spend an evening choosing.

11.5 Settings worth changing immediately

Show whitespace
Makes stray tabs visible. *Editor: Render Whitespace* → *boundary*.

Rulers at 80 and 100
A visual reminder to keep lines readable. `"editor.rulers": [80, 100]`.

Format on save
Consistent formatting, zero effort.

Font size
If you plan to film any of this, **16–18 pt minimum**. Nobody can read your normal size on a phone.

CHECKPOINT The editor understands your project

Open `ToolchainCheck.java` and:

1. Type `System.out.` — a completion list appears.
2. Delete a semicolon — a red squiggle appears immediately, before any build.
3. Put it back, then click *Run* above `main` — output appears in a panel.
4. Open `ToolchainCheckTest.java` and click *Run Test* — three green ticks.

If completions do not appear, the language server has not finished indexing, or it failed to import the Maven project. Reload the window and give it a minute.

PART IV

The Hardware

CHAPTER 12

Wiring the reader

GOAL Connect the RC522 and prove it answers.

CONCEPT Separating “is it the wiring?” from “is it the code?” — permanently.

YOU END WITH `WIRING OK. The RC522 is alive and talking.`

This is the last chapter of Volume Zero. When its checkpoint passes, you have a complete working environment and you can start writing code.

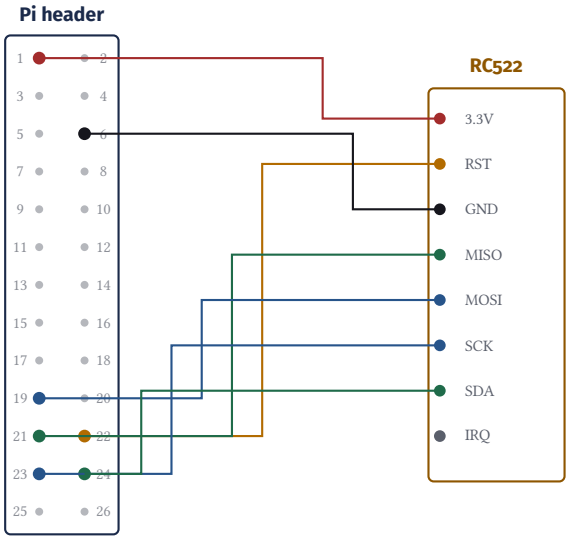
12.1 Power off first

```
ssh brewer 'sudo shutdown -h now'
```

Wait for the green LED to stop flickering, then unplug the power. Wiring a powered Pi is how you destroy a Pi.

12.2 The seven wires

RC522	Pi pin	Pi name	Note
3.3V	1	3V3	**Never 5 V**
RST	22	BCM 25	reset
GND	6	GND	
MISO	21	BCM 9	chip → Pi
MOSI	19	BCM 10	Pi → chip
SCK	23	BCM 11	clock
SDA	24	BCM 8 (CE0)	chip select
IRQ	—	—	leave unconnected



pin 1 is the corner nearest the SD card slot

TRAP 3.3 V, not 5 V

Pin 1 is 3.3 V. Pin 2 — *directly beside it* — is 5 V, and it will destroy the RC522. Count carefully. Pin 1 is the corner nearest the SD card slot; odd numbers run down one side, even down the other. Getting this wrong is silent, immediate and permanent.

TRAP SDA is not I²C here

The RC522 board labels its chip-select pin `SDA`, which is an I²C name. The module supports several buses and the silkscreen reflects a different one.
For SPI, `SDA` goes to **CE0, Pi pin 24**. This is the single most common mis-wiring, and its symptom is `VersionReg = 0x00`.

12.3 Check it twice

Before powering on, verify each wire by touching both ends, one at a time. Seven wires, seven checks. Two minutes now saves an hour later — and unlike code, you cannot `printf` a wire.

Then power up and wait a minute.

12.4 Probe it

```
# on your laptop
make deploy-full
make probe
```

VERIFICATION GATE

```
=== RC522 wiring probe ===
Opening SPI bus 0, chip-select 0, 1 MHz, mode 0...
VersionReg   (0x37) = 0x92
TxControlReg (0x14) = 0x80

  -> WIRING OK. The RC522 is alive and talking.
     You are ready for Chapter 1 of the main book.
```

`0x92` is the MFRC522 v2.0 identifier (`0x91` is v1.0 — also fine). The chip is powered, on the right pins, and speaking SPI at the right clock and mode.

12.5 If it does not say that

The probe diagnoses itself. Its three failure messages, and what they mean:

Reading	Meaning	Check, in order
`0x00`	MISO stuck low — the chip is not driving the line	Power (3.3 V and GND); then SDA → pin 24; then MISO → pin 21
`0xFF`	MISO floating — nothing connected	MISO → pin 21. If connected, suspect a dead module
varies	noise	**Shorter wires.** Run the probe three times: if the value *changes*, it is definitely wiring

And if it fails to open SPI at all:

Error	Fix
`No such file or directory`	SPI not enabled. `sudo raspi-config`, Interface Options, then **reboot**
`Permission denied`	Not in the `spi` group — and remember the change needs a fresh login
`No gpio chip found`	You ran it on your laptop, not the Pi

CONCEPT Why this probe exists at all

It answers one question and nothing else: *is the hardware wired correctly?*
From Volume One, Chapter 1 onwards you will be writing your own driver, and its bugs will look exactly like wiring faults. Having a known-good, disposable diagnostic means you never have to wonder which one you are looking at.
Keep it. Run it whenever anything hardware-shaped goes wrong. It is deliberately ugly — one flat method, no abstractions — because a diagnostic you have to debug is worthless.

12.6 You are done

CHECKPOINT Volume Zero complete

```
ssh brewer 'hostname'          # one word, no password
cd ~/midnight-brewer && mvn test    # Tests run: 3
make deploy && make run        # runs on the Pi, aarch64
make probe                     # WIRING OK
git log --oneline              # at least one commit
```

```
git push                        # backed up
```

Six commands, six green results. You have:

- a headless Pi reachable by name, with key-only SSH;
- Java 21 and Maven on both machines;
- a project that builds and tests in seconds;
- a five-second deploy loop to real hardware;
- your work in git and on GitHub;
- an NFC reader answering.

None of that was specific to NFC. It is the setup for any embedded Linux project you will ever do.

Now open **Volume One** — *Building a Contactless Payment Terminal From Scratch* — and turn to Chapter 1. You will write an interface, build a fake SPI bus, and test a driver with the reader unplugged.

The environment is no longer something that can be wrong.

PART V

Appendices

CHAPTER A

Cheat sheet

One page per topic. Photocopy, print, or keep the PDF open on a second screen.

A.1 The daily loop

```
mvn test                      # fast, local, no hardware
make deploy && make run       # onto the real Pi
make probe                    # is the reader still alive?
git add -A && git commit -m "feat: ..." && git push
```

A.2 SSH

```
ssh brewer                            # log in (alias from ~/.ssh/config)
ssh brewer 'uptime'                   # run one command, come straight back
ssh -v brewer                         # verbose -- diagnose auth failures
exit                                  # or Ctrl-D

scp file.txt brewer:~                 # copy a file over
scp -r brewer:~/logs ./logs           # copy a folder back
rsync -a --delete src/ brewer:dst/    # mirror a folder (note trailing slashes)
```

`~/.ssh/config`

```
Host brewer
    HostName brewer.local
    User martin
    IdentityFile ~/.ssh/id_ed25519
    ServerAliveInterval 30
```

```
    ServerAliveCountMax 6
```

Key setup, from scratch

```
ssh-keygen -t ed25519 -C "martin@laptop"    # once, ever
ssh-copy-id martin@brewer.local             # once per machine
chmod 700 ~/.ssh && chmod 600 ~/.ssh/*      # if keys are ignored
```

A.3 Git

```
git status                    # what changed?
git diff                      # show the actual changes
git diff --staged             # ... that are staged
git add -A                    # stage everything
git add -p                    # stage interactively, hunk by hunk
git commit -m "feat: ..."     # save point
git push                      # back it up
git log --oneline -10         # last ten commits
git log --oneline -- path/to/File.java     # history of one file
git show HEAD                 # what was in the last commit
git checkout -- file.java     # DISCARD my changes to this file
git reset --soft HEAD~1       # undo the last commit, keep the changes
git stash / git stash pop     # shelve work temporarily
```

Message prefixes

```
feat:   a new capability
fix:    a bug fixed
chore:  housekeeping, build, dependencies
docs:   documentation
test:   tests only
```

A.4 Maven

```
mvn test                      # compile + run tests
```

```
mvn -q compile                # compile quietly
mvn clean                     # delete target/
mvn clean test                # when you suspect stale output
mvn test -Dtest=FooTest       # one test class
mvn dependency:copy-dependencies -DincludeScope=runtime
mvn dependency:tree           # why is THAT jar on my classpath?
```

Running things

```
java -cp target/classes com.midnightbrewer.ToolchainCheck
java -cp "target/classes:target/dependency/*" com.midnightbrewer.diag.SpiProbe
```

A.5 The Pi

```
# identity and health
hostname
cat /proc/device-tree/model; echo
free -h                       # memory
df -h /                       # disk
vcgencmd measure_temp         # temperature
uptime                        # load and how long it has been up

# hardware
ls -l /dev/spidev0.0          # SPI device present?
groups                        # am I in spi and gpio?
lsmod | grep spi              # SPI kernel modules loaded

# services and logs
sudo systemctl status ssh
journalctl -u ssh -n 50       # last 50 lines of a service's log
journalctl -f                 # follow everything, live
dmesg | tail -20              # kernel messages (undervoltage shows here)

# packages
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y <package>

sudo reboot
sudo shutdown -h now          # ALWAYS before touching wiring
```

A.6 Terminal

```
pwd                     # where am I?
ls -la                  # what is here, including hidden
cd -                    # back to the previous directory
find . -name "*.java"   # find files by name
grep -rn "TODO" src/    # search a tree, with line numbers
chmod +x script.sh      # make executable
du -sh *                # what is taking up space
history | grep ssh      # what was that command last week?
```

Tab	complete a name (and prove it exists)
Up	previous command
Ctrl-R	search history
Ctrl-C	stop the running program
Ctrl-D	end input / log out
Ctrl-A / Ctrl-E	start / end of line

A.7 Pin reference

RC522	Pi pin	BCM	Function
3.3V	1	—	power (**never pin 2**)
GND	6	—	ground
MOSI	19	10	Pi → chip
MISO	21	9	chip → Pi
RST	22	25	reset
SCK	23	11	clock
SDA	24	8	chip select (CE0)
IRQ	—	—	not connected

A.8 Values worth memorising

`0x91` / `0x92`	healthy RC522 version byte
`0x00`	MISO stuck low — power or wiring
`0xFF`	MISO floating — not connected
`aarch64`	correct Pi architecture (64-bit)
`Tests run: 3`	the starter pack's test count

CHAPTER B

When it does not work

Indexed by **what you see on screen**. Search for your error message here before searching the internet — almost every setup failure has a known cause and a one-line fix.

B.1 First, three questions

Before diagnosing anything, answer these. They resolve most problems on their own.

1. **Which machine am I on?** Run `hostname`. Half of all setup confusion is the right command on the wrong machine.
2. **Where am I?** Run `pwd`. The other half is the right command in the wrong directory.
3. **Did I reboot / log out?** SPI needs a reboot. Group membership needs a fresh login. Both fail silently until you do.

B.2 Cannot reach the Pi

You see	Cause / fix
`ssh: Could not resolve hostname brewer.local`	mDNS not working. Find the IP from your router and use it directly; then put the IP in `HostName` in `~/.ssh/config`.
`ssh: connect to host ... port 22: Connection refused`	The Pi is up but SSH is not running — you did not enable it in Imager. Re-flash.
`... Operation timed out`	The Pi is not on the network at all. Wi-Fi credentials, 5 GHz, wireless country, or power.
`ping` works, `ssh` does not	SSH not enabled. Re-flash with the Services tab ticked.

You see	Cause / fix
Never appeared at all	Re-flash rather than diagnose. Five minutes, and it fixes most cases.
Worked yesterday, not today	DHCP gave it a new address. Use the hostname, or set a static lease in your router.

`REMOTE HOST IDENTIFICATION HAS CHANGED`

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

You reflashed the Pi (new host key), or your router gave that address to a different device. Remove the stale entry:

```
ssh-keygen -R brewer.local
ssh-keygen -R 192.168.1.42      # also the IP, if you used it
```

> **TRAP Never disable host checking to make this go away**
>
> `StrictHostKeyChecking=no` silences the warning permanently and removes the only protection you have against connecting to an impostor.
> Remove the *specific* stale entry instead. It is one command.

B.3 SSH keys still ask for a password

You see	Cause / fix
Asks for a password after `ssh-copy-id`	Permissions. On the Pi: `chmod 700 ~/.ssh` and `chmod 600 ~/.ssh/authorized_keys`. SSH ignores loose permissions *silently*.
Works for `ssh`, not for `scp`/`rsync`	Those read `~/.ssh/config` too — check the `Host` alias matches what you are typing.
`Permission denied (publickey)` after disabling passwords	The key was never installed correctly. If you can no longer get in, mount the SD card on your laptop and re-enable `PasswordAuthentication`, or re-flash.

You see	Cause / fix
Prompts for the key passphrase every time	Add it to the agent: `ssh-add ~/.ssh/id_ed25519` (macOS: `ssh-add -apple-use-keychain`).

Diagnose with `ssh -v brewer` — it prints which keys were offered and what the server did with them.

B.4 Java and Maven

You see	Cause / fix
`java: command not found`	Not installed, or not on your `PATH`. Reopen your terminal after installing.
`mvn -v` reports a different Java than `java -version`	Maven uses `JAVA_HOME`. Set it: `export JAVA_HOME=$(/usr/libexec/java_home -v 21)` on macOS, or `export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-arm64` on the Pi.
`class file has wrong version 65.0`	Compiled with a newer Java than you are running. Check both machines are 21.
`Tests run: 0`	Tests not found. Class must end in `Test` and live under `src/test/java`.
`Non-parseable POM`	XML error. A comment containing – is the usual culprit; it is illegal inside `<!- ->`.
Build downloads forever	First build only. If it repeats, delete `~/.m2/repository` and try once more.
`ClassNotFoundException` at runtime	Classpath missing the dependency jars. Use `-cp "target/classes:target/dependency/*"` and run `make deploy-full`.

B.5 SPI and the reader

You see	Cause / fix
`/dev/spidev0.0: No such file`	SPI not enabled, or you did not reboot after enabling it.
`Permission denied` on spidev	Not in the `spi` group — *or* you are, but have not logged out and back in since.
`No gpio chip found`	You are running on your laptop, not the Pi.
`VersionReg = 0x00`	MISO stuck low. Power (3.3 V, not 5 V), then SDA → pin 24, then MISO → pin 21.
`VersionReg = 0xFF`	MISO floating. Not connected, or dead module.
Version changes between runs	Wire length. **Shorten the jumpers.** This is not a code problem.
Device or resource busy	A previous run is still holding SPI. `ssh brewer 'pkill java'`. Use `run.sh` (which passes `ssh -t`) so Ctrl-C actually reaches it.

B.6 The Pi misbehaves

You see	Cause / fix
Random reboots, or a rainbow square	Undervoltage. Replace the power supply *and* the cable. Confirm with `dmesg | grep -i voltage`.
SSH dies during a build	Out of memory — the kernel killed `sshd`. Add swap (Chapter 7) and build on your laptop.
Everything is extremely slow	A slow or failing SD card. Check with `dmesg | grep -i mmc`.
`No space left on device`	`df -h /`. Clear caches with `sudo apt clean` and delete old `target/` folders.
Filesystem is read-only	The card is failing or was corrupted by a power cut. Back up and re-flash.

B.7 Git and GitHub

You see	Cause / fix
`Permission denied (publickey)` on push	Your public key is not on GitHub. Test with `ssh -T git@github.com`.
`Support for password authentication was removed`	You are using an HTTPS remote. Switch to SSH: `git remote set-url origin git@github.com:USER/REPO.git`.
`failed to push some refs`	The remote has commits you do not. `git pull --rebase` then push.
`Please tell me who you are`	Set `user.name` and `user.email` — see Chapter 2.
Stuck in a full-screen editor	That is vim. Press Escape, type `:q!`, press Enter. Then `git config --global core.editor nano`.
Committed a secret	Assume it is compromised. Rotate the key or password. Deleting it in a later commit does not remove it from history.

B.8 The deploy loop

You see	Cause / fix
Changes have no effect	Stale classes. `mvn clean compile && make deploy-full`.
`rsync: command not found`	Install it on *both* machines.
Files land in `classes/classes/`	Missing trailing slash on the rsync source. `target/classes/` means "the contents of".
Ctrl-C does not stop the remote program	Use `ssh -t`. `run.sh` already does.
`make: command not found`	macOS: install the Xcode command line tools. Linux: `sudo apt install make`.

B.9 Still stuck?

Narrow it down before asking anyone, including a search engine:

1. **Does it work on the other machine?** If `mvn test` passes on your laptop but fails on the Pi, the problem is environmental, not code.
2. **Did it ever work?** If yes, `git diff` shows exactly what changed since.

3. **What is the smallest thing that fails?** `ssh brewer 'echo hi'` before `make deploy`. Test one layer at a time.
4. **Read the whole error.** The useful line is often the third, not the first — and in Java the useful part is usually `Caused by:` near the bottom.

> **CONCEPT Bisect, do not stare**
>
> The habit that separates fast debugging from slow: instead of reading code hoping to spot the fault, **find the boundary between what works and what does not**.
> `ssh` works but `scp` does not → it is the config, not the network. `mvn test` passes but `make run` fails → it is the classpath, not the code.
> Each check halves the search space. Four checks turn sixteen possibilities into one.